ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| ассистент | | В. В. Жукалин |
|---|---|---|
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

# УСТАНОВКА POSTGRESQL И ПОДКЛЮЧЕНИЕ PRISMA ORM

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

| СТУДЕНТ гр. № 4321 | | И.А. Лаврешин |
|---|---|---|
| | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2026

## 1 Цель работы

Установить и проверить локальный PostgreSQL, создать изолированную роль и базу данных SignalBoard, подключить Prisma ORM 6.19.2 к существующему Next.js-проекту, безопасно настроить строку подключения, сгенерировать Prisma Client и выполнить диагностический запрос к серверу базы данных.

## 2 Задание

Установить поддерживаемую версию PostgreSQL с официальной страницы загрузки, сохранить сведения о компонентах, порте и каталоге данных, проверить запуск службы, освоить подключение через psql. Создать отдельную роль signal_app без прав суперпользователя и базу данных signalboard с владельцем signal_app, проверить подключение под созданной ролью. Установить одинаковые версии prisma и @prisma/client, инициализировать Prisma для провайдера postgresql, настроить DATABASE_URL через переменную окружения и исключить секреты из системы контроля версий. Описать generator, datasource и временную модель проверки, проверить схему командами prisma format и prisma validate, сгенерировать Prisma Client, создать единственный экземпляр PrismaClient, выполнить диагностический SQL-запрос из TypeScript и проверить npm run lint и npm run build. Предметные таблицы в этой работе не создаются.

**3 Ход выполнения задания**

Среда выполнения: Windows 11 Pro (версия 10.0.26200), Node.js v24.20.0, npm 11.19.0, PostgreSQL *[версия, например 17.x]*. При установке выбраны компоненты PostgreSQL Server и Command Line Tools, порт *[5432]*, каталог данных *[C:\Program Files\PostgreSQL\NN\data]*, служба Windows *[postgresql-x64-NN]*. Пароль суперпользователя postgres в отчёт не включается.

Основные понятия, которые важно различать:

– сервер PostgreSQL – запущенный процесс (служба Windows), который принимает подключения на порту и обслуживает один кластер;

– кластер – набор баз данных в одном каталоге данных под управлением одного сервера;

– база данных – изолированный набор схем и объектов; подключение всегда выполняется к конкретной базе (signalboard);

– схема – пространство имён внутри базы; по умолчанию используется public;

– роль – учётная запись с правами; приложение работает под signal_app, а не под суперпользователем postgres;

– клиент – программа, которая подключается к серверу: psql, Prisma Client, pgAdmin.

Роль и база созданы в psql под пользователем postgres. Пароль в листинге заменён заполнителем:

```
CREATE ROLE signal_app
  WITH LOGIN
  PASSWORD '***';

CREATE DATABASE signalboard
  WITH OWNER = signal_app
       ENCODING = 'UTF8';
```

Роль signal_app создана без атрибутов SUPERUSER, CREATEDB и CREATEROLE: если приложение будет скомпрометировано, оно не сможет управлять другими базами и ролями сервера. Подключение под созданной ролью проверено командой psql -h localhost -p 5432 -U signal_app -d

signalboard; метакоманда \conninfo показывает базу, роль, хост и порт текущего соединения.

*[ВСТАВИТЬ СКРИНШОТ: окно терминала с выводом psql --version]*

Рисунок 1 – Результат psql --version

*[ВСТАВИТЬ СКРИНШОТ: psql -U signal_app -d signalboard: SELECT current_database(), current_user, inet_server_port(); и \dt]*

Рисунок 2 – Прикладной запрос под ролью signal_app

В проект установлены prisma (devDependency) и @prisma/client одной версии 6.19.2 – версии эталонного проекта курса. Совпадение важно: CLI генерирует клиент под свою версию движка. Вывод npx prisma --version приведён на рисунке 3.

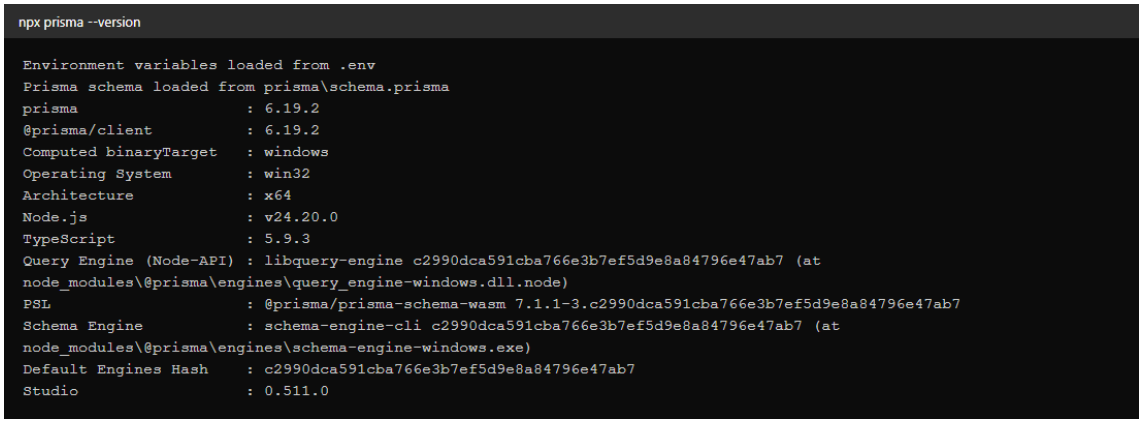



Рисунок 3 – Вывод npx prisma --version

Дерево новых файлов проекта показано на рисунке 4.

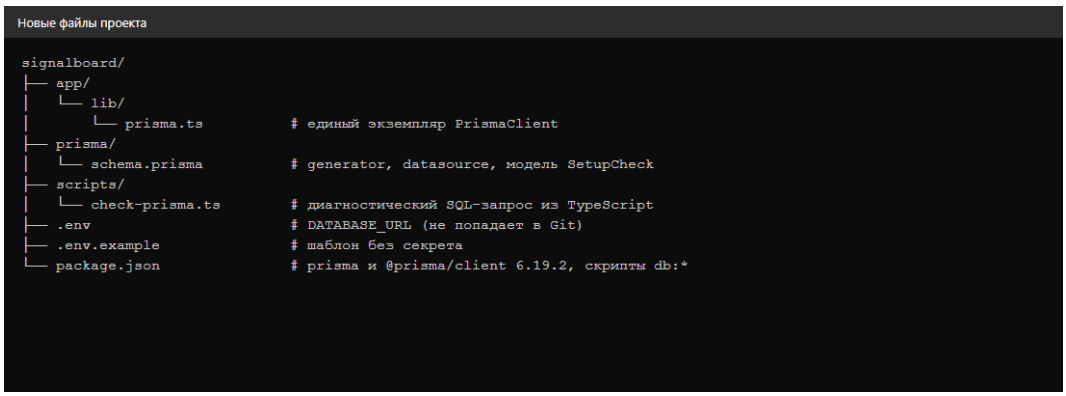



Рисунок 4 – Новые файлы проекта ЛР5

Файл prisma/schema.prisma содержит блок generator (генерация prisma-client-js), блок datasource (провайдер postgresql и строка подключения из переменной окружения DATABASE_URL) и временную модель SetupCheck, нужную только для того, чтобы генерация клиента прошла. Секретов в схеме нет:

```
// Генерирует Prisma Client для установленной версии Prisma ORM 6.19.2.
generator client {
  provider = "prisma-client-js"
}

// Читает локальное подключение PostgreSQL из DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Временная модель позволяет проверить генерацию клиента.
// В ЛР6 она удаляется до создания первой миграции.
model SetupCheck {
  // Описывает тестовый первичный ключ только на уровне Prisma Schema.
  id Int @id
}
```

Строка подключения хранится в файле .env, который указан в .gitignore и не попадает в репозиторий. В репозиторий попадает только .env.example с заполнителем CHANGE_ME вместо пароля:

```
# Документирует формат подключения без настоящего пароля.
DATABASE_URL="postgresql://signal_app:CHANGE_ME@localhost:5432/signalboard?sc
hema=public"
NEXT_PUBLIC_YANDEX_MAPS_API_KEY=
```

Переменная не имеет префикса NEXT_PUBLIC_, поэтому Next.js не встраивает её в клиентский бандл. Если пароль содержит символы @, :, /, # или %, их нужно закодировать percent-encoding (например, @ → %40), иначе URL будет разобран неверно.

Файл app/lib/prisma.ts создаёт единственный экземпляр PrismaClient. В режиме разработки Next.js при горячей перезагрузке заново выполняет модули, и без сохранения в globalThis каждый раз создавался бы новый клиент с собственным пулом соединений, что быстро исчерпало бы лимит подключений PostgreSQL:

```
// Создаёт единый серверный экземпляр PrismaClient для Next.js.

import { PrismaClient } from "@prisma/client";

// Расширяем тип globalThis полем, используемым только в development.
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

// Переиспользуем клиент после горячей перезагрузки или создаём его впервые.
export const prisma = globalForPrisma.prisma ?? new PrismaClient();

// Сохраняем экземпляр глобально только вне production.
if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

Скрипт scripts/check-prisma.ts выполняет диагностический запрос SELECT current_database(), current_user, inet_server_port() через prisma.$queryRaw и выводит только безопасные сведения, не печатая DATABASE_URL. Запуск – npm run db:check.

Схема проверена командами prisma format и prisma validate (рисунок 5), клиент сгенерирован командой prisma generate (рисунок 6).

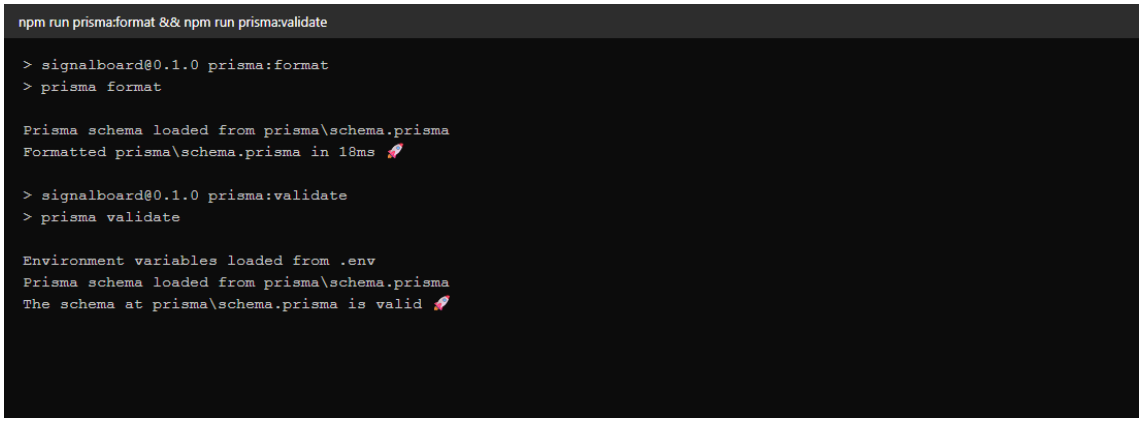



Рисунок 5 – Успешные prisma format и prisma validate

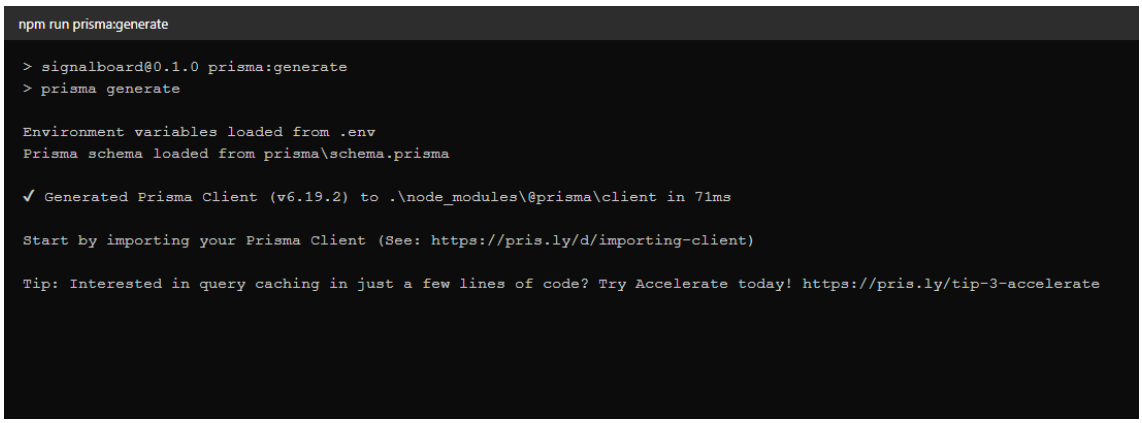



Рисунок 6 – Успешный prisma generate

*[ВСТАВИТЬ СКРИНШОТ: вывод «Prisma connected: { database: 'signalboard', role: 'signal_app', port: 5432 }»]*

Рисунок 7 – Успешный TypeScript-запрос npm run db:check

Миграции в этой работе не выполняются: команды prisma migrate dev и prisma db push не запускались, предметные таблицы будут спроектированы в лабораторной работе № 6. Поэтому \dt в базе signalboard сообщает, что отношений нет, и это ожидаемый результат.

Возникшая проблема и её диагностика. При первом запуске npm run db:check служба PostgreSQL не была запущена, и скрипт завершился ошибкой PrismaClientInitializationError «Can't reach database server at localhost:5432» (код P1001). Диагностика: P1001 означает, что сервер недоступен по сети – нужно проверить, запущена ли служба (services.msc или Get-Service postgresql*), совпадают ли хост и порт в DATABASE_URL и не блокирует ли порт брандмауэр. Если бы сервер ответил, но отверг роль или пароль, была бы ошибка P1000. Решение – запустить службу PostgreSQL и сверить порт, после чего скрипт выполняется успешно. Вывод ошибки приведён на рисунке 8; строка подключения в нём не раскрывается.

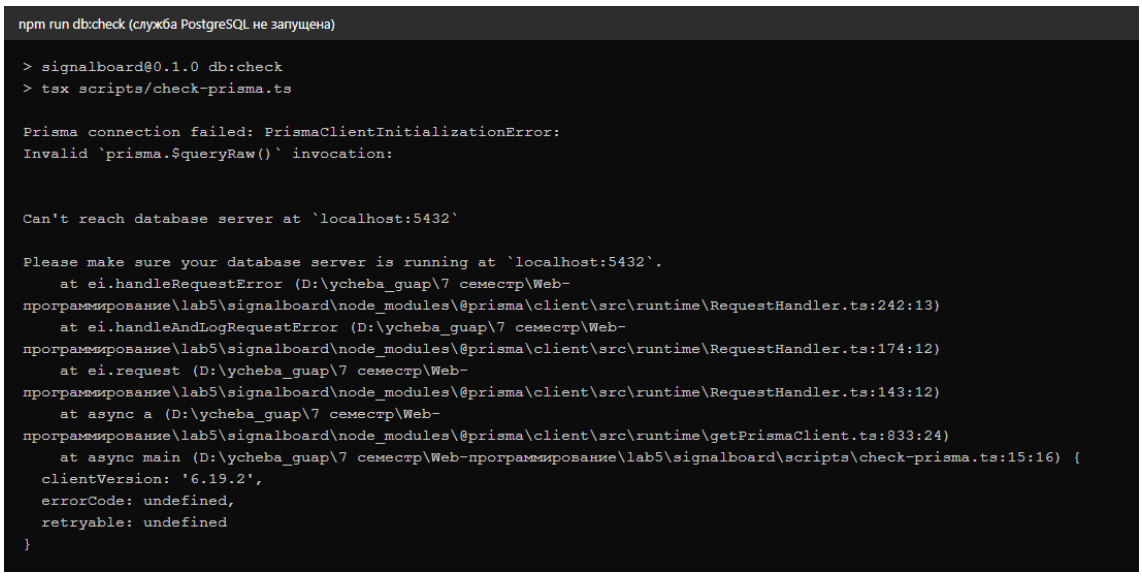



Рисунок 8 – Ошибка P1001 при остановленной службе PostgreSQL

Проверка кода: npm run lint и npm run build завершились без ошибок (рисунки 9–10). Файл app/lib/prisma.ts не импортируется клиентскими компонентами, поэтому Prisma Client не попадает в браузерный бандл.

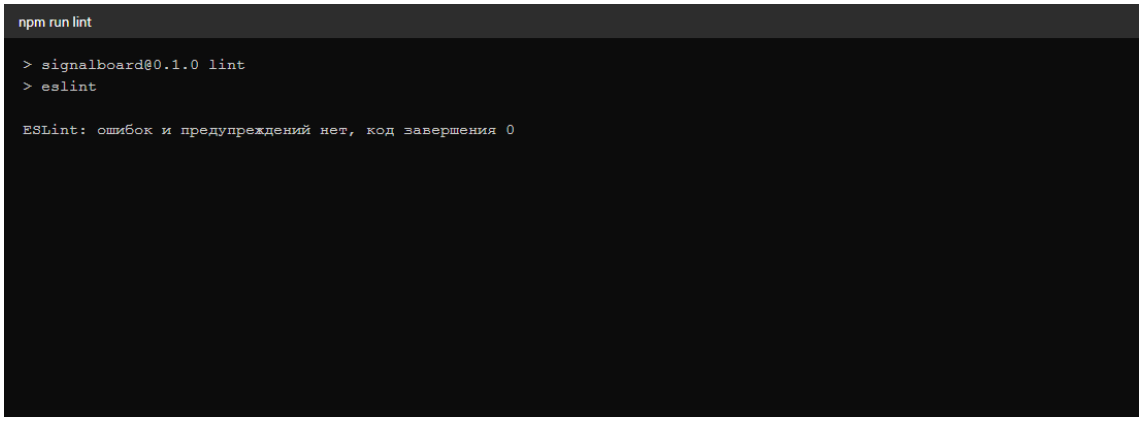



Рисунок 9 – Результат npm run lint

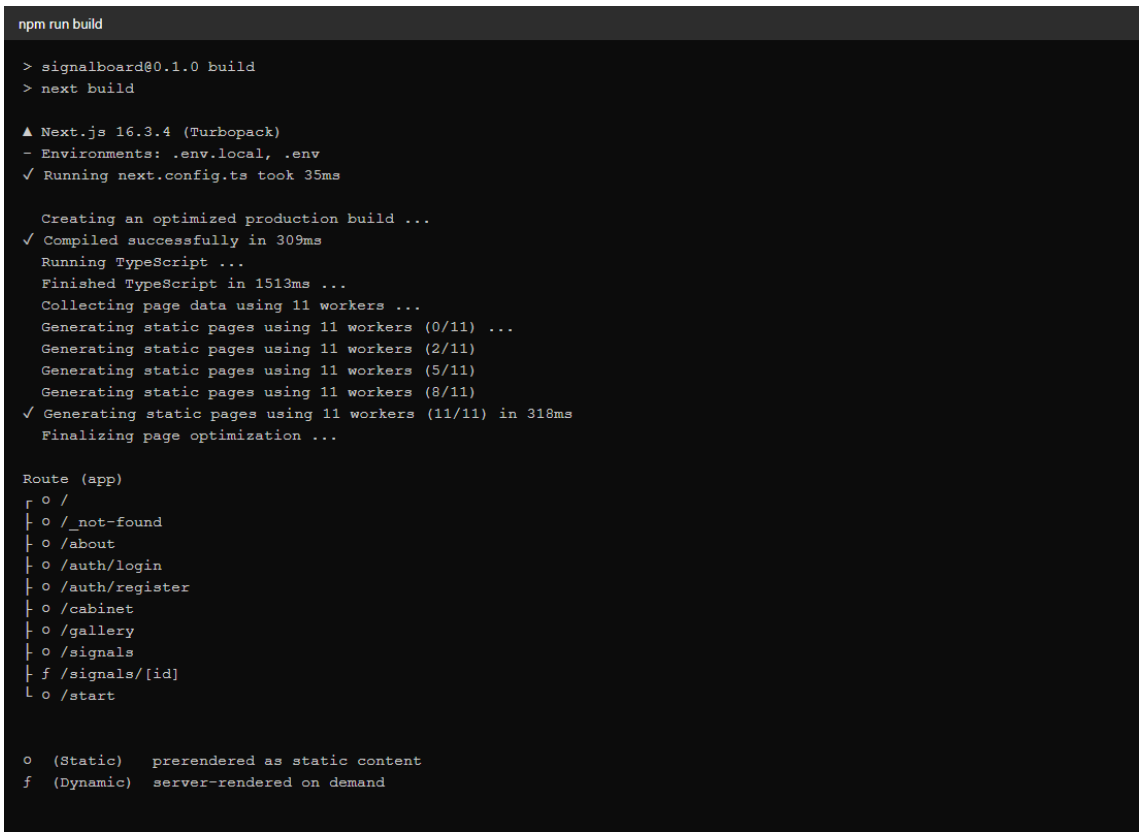



Рисунок 10 – Результат npm run build

## 4 Вывод

В ходе работы установлен локальный PostgreSQL, созданы роль signal_app без прав суперпользователя и база signalboard, к проекту SignalBoard подключена Prisma ORM 6.19.2 с одинаковыми версиями prisma и @prisma/client. Строка подключения вынесена в .env, исключённый из Git, а .env.example содержит только заполнитель. Схема проходит prisma format и prisma validate, клиент генерируется, единый экземпляр PrismaClient хранится в globalThis в режиме разработки, диагностический скрипт проверяет фактические базу, роль и порт соединения. Предметные таблицы не создавались. Проект проходит npm run lint и npm run build.

# ПРИЛОЖЕНИЕ А

## Листинг с кодом программы

### Файл prisma/schema.prisma

```
// Генерирует Prisma Client для установленной версии Prisma ORM 6.19.2.
generator client {
  provider = "prisma-client-js"
}

// Читает локальное подключение PostgreSQL из DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Временная модель позволяет проверить генерацию клиента.
// В ЛР6 она удаляется до создания первой миграции.
model SetupCheck {
  // Описывает тестовый первичный ключ только на уровне Prisma Schema.
  id Int @id
}
```

### Файл app/lib/prisma.ts

```
// Создаёт единый серверный экземпляр PrismaClient для Next.js.

import { PrismaClient } from "@prisma/client";

// Расширяем тип globalThis полем, используемым только в development.
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

// Переиспользуем клиент после горячей перезагрузки или создаём его впервые.
export const prisma = globalForPrisma.prisma ?? new PrismaClient();

// Сохраняем экземпляр глобально только вне production.
if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

### Файл scripts/check-prisma.ts

```
// Проверяет соединение Prisma с PostgreSQL без создания предметных таблиц.

import "dotenv/config";
import { prisma } from "../app/lib/prisma";

// Описываем форму строки, которую возвращает диагностический SELECT.
type ConnectionInfo = {
  database: string;
  role: string;
  port: number;
};

async function main() {
  // Запрашиваем у сервера фактические параметры текущего соединения.
  const rows = await prisma.$queryRaw<ConnectionInfo[]>`
    SELECT
      current_database() AS database,
```

```
      current_user AS role,
      inet_server_port() AS port
  `;

  // Выводим только безопасные диагностические сведения, без DATABASE_URL.
  console.log("Prisma connected:", rows[0]);
}

// Завершаем процесс с ненулевым кодом при ошибке подключения.
main()
  .catch((error) => {
    console.error("Prisma connection failed:", error);
    process.exitCode = 1;
  })
  .finally(async () => {
    // Освобождаем ресурсы отдельного диагностического скрипта.
    await prisma.$disconnect();
  });
```

## Файл .env.example

```
# Документирует формат подключения без настоящего пароля.
DATABASE_URL="postgresql://signal_app:CHANGE_ME@localhost:5432/signalboard?sc
hema=public"
NEXT_PUBLIC_YANDEX_MAPS_API_KEY=
```

## Файл package.json

```
{
  "name": "signalboard",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "prisma:format": "prisma format",
    "prisma:validate": "prisma validate",
    "prisma:generate": "prisma generate",
    "db:check": "tsx scripts/check-prisma.ts"
  },
  "dependencies": {
    "@prisma/client": "^6.19.2",
    "dotenv": "^17.4.2",
    "next": "16.3.4",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  },
  "devDependencies": {
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.3.4",
    "prisma": "^6.19.2",
    "tsx": "^4.23.13",
    "typescript": "^5"
  }
}
```